

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

Факультет прикладной математики, информатики и механики

Кафедра вычислительной математики и информационных
прикладных технологий

Направление 01.04.02 Прикладная математика и информатика

Отчёт

по производственной практике (научно-исследовательская работа)

Тема	Разработка аппаратной реализации 2D графического ускорителя
Отчетный семестр	4

Обучающийся	2 курс, 11 группа, Старухин Д.М.
Руководитель от кафедры	к.ф.-м.н., доц. Медведева О.А.

Воронеж 2026

Содержание

Введение	3
Постановка задачи	4
Актуальность поставленной задачи	5
1. Обзор архитектур графических ускорителей	6
1.1. Обзор архитектур графических ускорителей компании Nvidia	6
2. Классификация параллельных вычислительных моделей (SIMT vs SIMD, MIMD).	12
3. Архитектура ядра	15
Заключение	16
Литература	17

Введение

В этой работе будет представлена аппаратная архитектура реализации ядра для графических ускорителей. Также, в этой работе будут рассмотрены другие архитектуры графического ядра у ведущих компаний в разработке графических ускорителей. С учётом уже существующих архитектур графических ядер, в работе будет представлена собственная архитектура графического ядра.

Постановка задачи

Компанией АО «ПКК» Миландр была поставлена задача разработать аппаратную реализацию 2D графического ускорителя. Перед началом аппаратной разработки необходимо описать архитектуру, которой должен соответствовать сложно-функциональный (СФ) блок.

При разработке архитектуры учитываются проделанные ранее обзор и анализ 2D графических ускорителей [1]. Разработка ведется с учётом предложенной ранее [2] архитектурой графического ускорителя.

Перед тем как разработать аппаратную реализацию 2D графического ускорителя, необходимо и важно разобраться в предназначении и структуре различных компонентов IP-блока. Одним из таких компонентов, является графическое ядро.

Графическое ядро имеет довольно сложную архитектуру, которое состоит из элементов, которые должны корректно соответствовать друг другу. Поэтому, перед разработкой графического ядра, нужно разработать его архитектуру, с учётом разработок графических ядер других кампаний.

Актуальность поставленной задачи

В современном мире, в условиях развития нейросетей и импортозамещения, создание собственных специализированных графических и вычислительных архитектур является довольно важной задачей.

Нейросети все больше вовлекаются во многие аспекты, начиная от развлекательной индустрии, заканчивая научными исследованиями. Искусственный интеллект образовал огромный спрос видеокарт, с помощью которых можно собрать вычислительный сервер, на которых, можно развернуть нейросеть для решения различных задач. Видеокарта состоит из большого количества элементов, одним из которых является графический процессор [3]. графический процессор обладает большим количеством ядер, которые выполняют огромный объем вычислений с плавающей точкой.

Помимо искусственного интеллекта, графические ядра также могут использоваться в вычислениях рендеринга графики, обработки видео или фото, а также в различных научных вычислениях, где требуется рассчитывать тысячи параллельных математических вычислений.

Разработка архитектуры графического ядра позволит снизить зависимость от зарубежных архитектур, таких, как AMD, ARM Mali и Nvidia. Благодаря разработке IP-блока, можно достичь создания доверенных систем, которые не будут содержать в себе наличия аппаратных «закладок» и недокументированных функций в критически важной инфраструктуре.

1. Обзор архитектур графических ускорителей

Перед тем, как заниматься разработкой архитектуры графического ядра, стоит обратить внимание на уже существующие разработки различных компаний.

1.1. Обзор архитектур графических ускорителей компании

Nvidia

Одна из компаний, занимающаяся разработкой графических ускорителей является Nvidia, которая с 1993 года занимается разработкой аппаратных ускорителей вычисления [4]. Nvidia за более 30 лет разработала большое количество микроархитектур для графических процессоров.

Одной из первых архитектур, разработанных Nvidia являлась архитектура Rankie, которая использовалась с 2003 по 2005 года в графических процессорах. Одним из графических процессоров, разработанным на микроархитектуре Rankie являлся NVIDIA NV30. Он обладал 125 миллионами транзисторами с техпроцессом 130нм. Графический процессор на тот момент обладала 4 пиксельными шейдерами, 3 вершинных шейдера, 8 блоков наложения текстур и 4 блоками вывода рендеринга (ROP (Render Output Unit)) [5].

Блок диаграмма архитектуры графического процессора NVIDIA NV30 представлена на рис. 1:

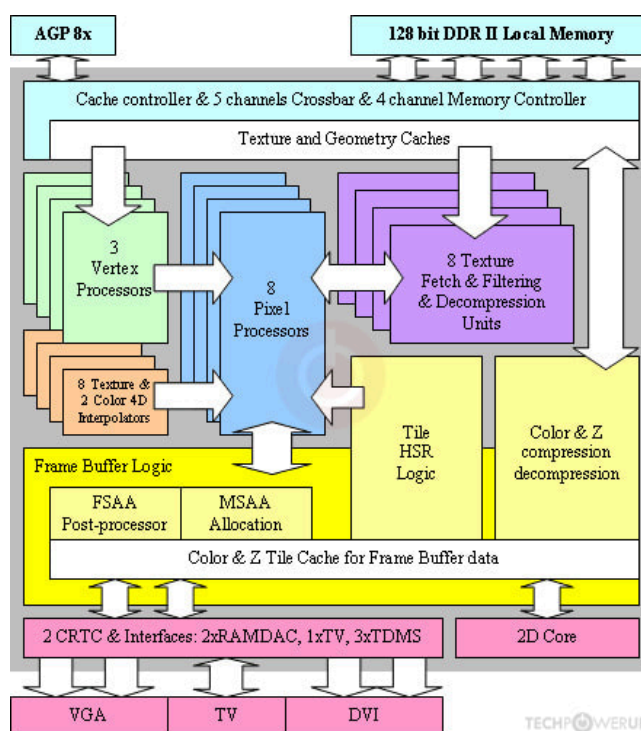


Рис. 1 — Блок схема архитектуры графического процессора NVIDIA NV30

По рис. 1 заметно, что архитектура очень похожа на современные 2D-графические ускорители, разрабатываемые в микроэлектронике. Понятие графического ядра тогда не существовало, вместо графического ядра все задачи выполняли вершинные и пиксельные процессоры. До 2006 года видеокарты использовали жестко разделенные процессоры, которые выполняли специализированные задачи. Но на сегодняшний день, вершинным и пиксельным процессорам пришли на замену унифицированные шейдерные ядра. Такие ядра выдавали более высокую производительность чем шейдеры, в последствии чего, будущие архитектуры графических процессоров стали основываться на использовании большого количества универсальных блоков. Такие блоки стали называться потоковыми процессорами (SP) [6].

Первая архитектура, которая стала использовать SP, как основу в графических процессорах стала архитектура Tesla, представленная с 2006 года. Данная архитектура в первую очередь отличалась от архитектуры Rarke использованием SP блоков, что являлось в новинку в архитектурах графических процессоров компании Nvidia. Архитектура графического процессора изображена на рис. 2:

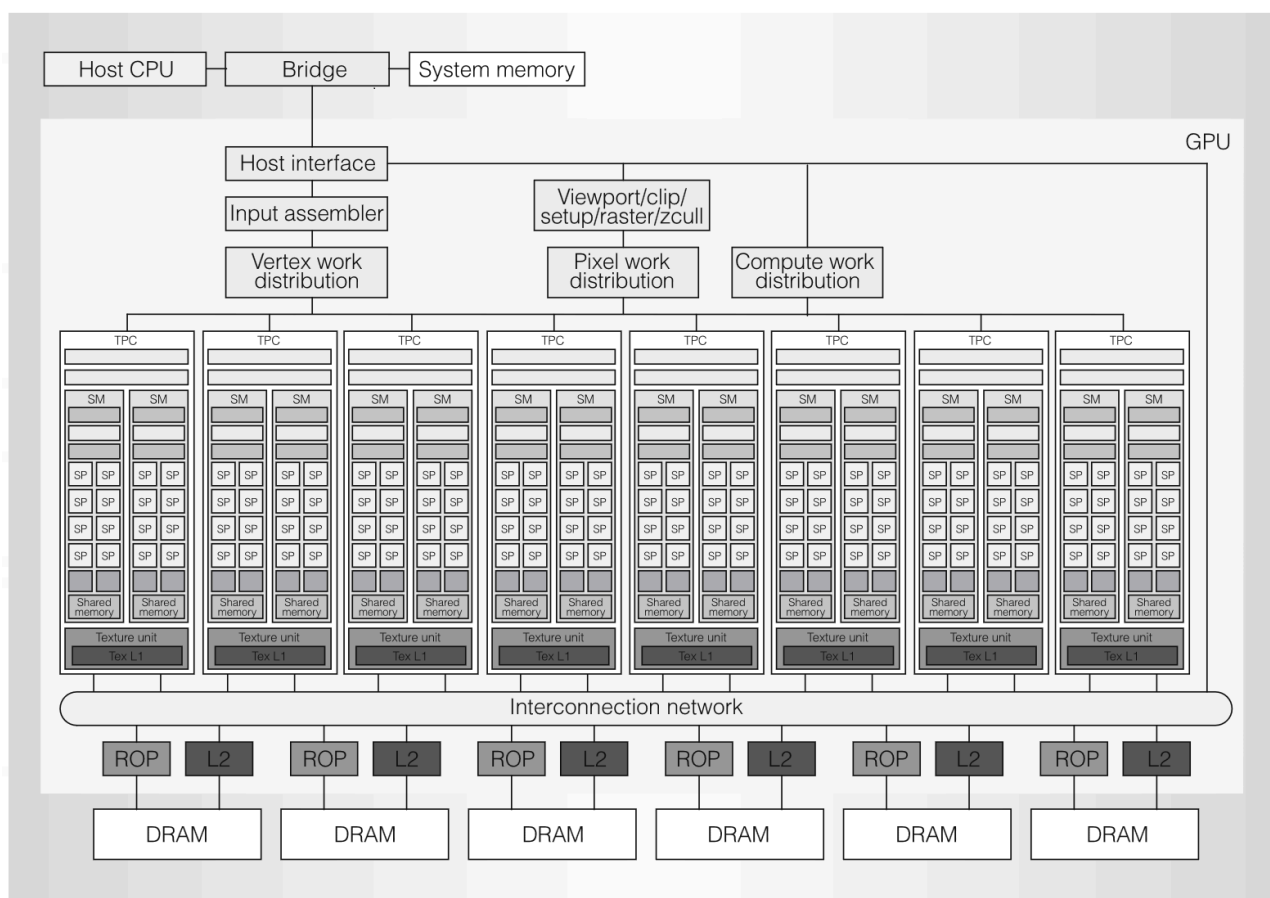


Рис. 2 — Архитектура графического процессора Tesla

Можно заметить, что архитектура Tesla сильно различается от Ranki. Теперь всю работу выполняют кластеры обработки текстур (TPC - texture/processor cluster). В графическом процессоре находится множество TPC и каждый из них является комплексным компонентом в архитектуре. Компонент TPC, в свою очередь, состоит из геометрического контроллера (Geometric controller), текстурного блока (Texture unit) контроллера потокового мультимикропроцессора (SMC) и самих потоковых мультимикропроцессоров (SM). Каждый потоковый мультимикропроцессор представляет собой набор из кешей (I cache и C cache), обработчика прерываний (MT issue), Блок специальных функций (SFU), общая память и потокового процессора (SP) [7]. Архитектура TPC представлена на рис. 3:

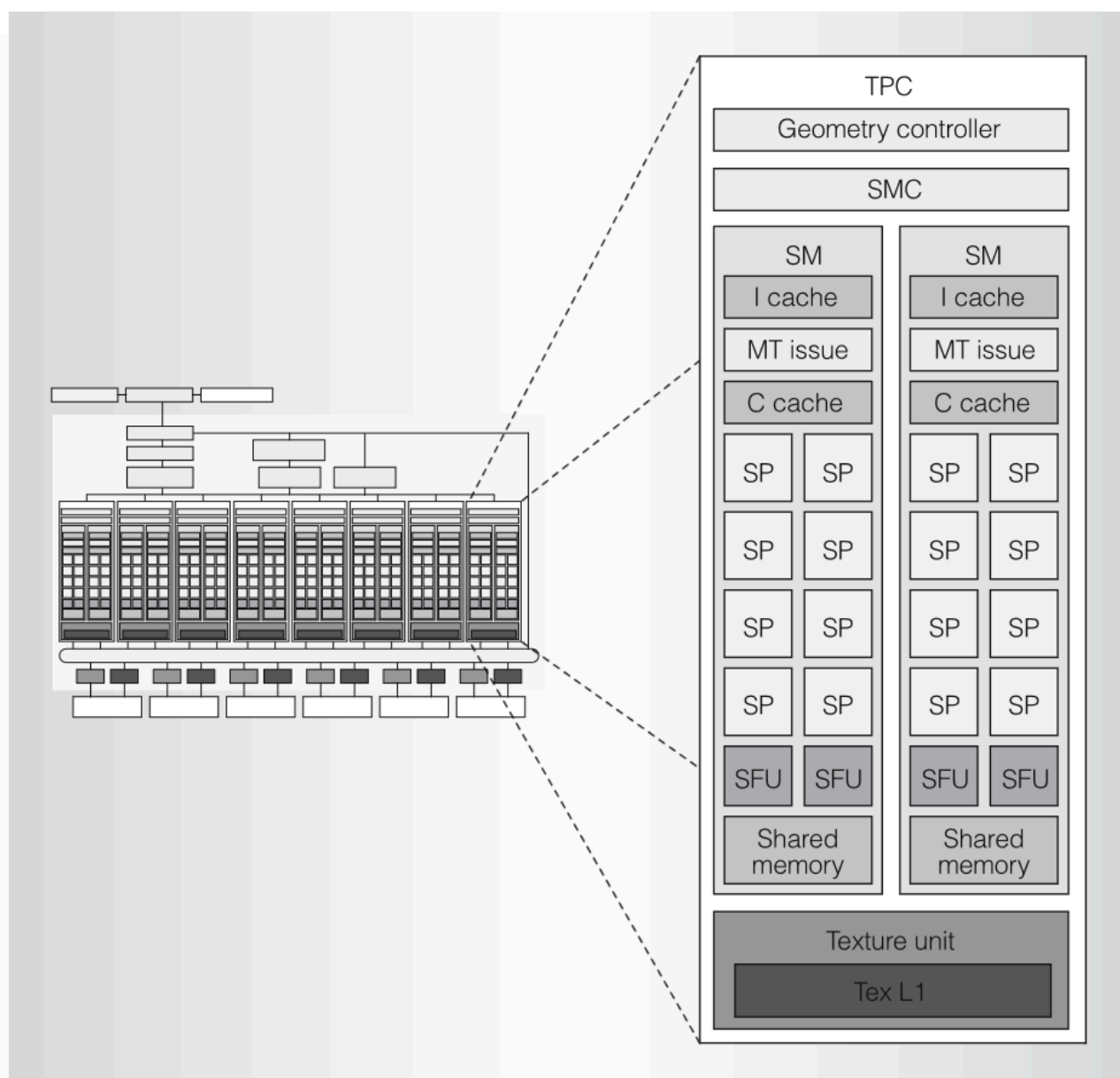


Рис. 3 — Архитектура графического процессора Tesla

По своей сути, основную работу с вычислениями выполняли SP, все сложные функции и операции, такие как вычисления синуса, косинуса или экспоненты отдавались на вычисление блоку SFU. В следующих архитектурах компании Nvidia, SP заменяются на CUDA-ядра, которые будут являться архитектурой параллельных вычислений и основой для разработки графических процессоров. Архитектура, в которой появились CUDA-ядра является архитектурой Fermi, используемая в видеокартах, начиная с 2010 года. Одной из главных особенностей стало появление CUDA-ядер. На рис. 4 можно увидеть компонент SM в архитектуре Fermi:

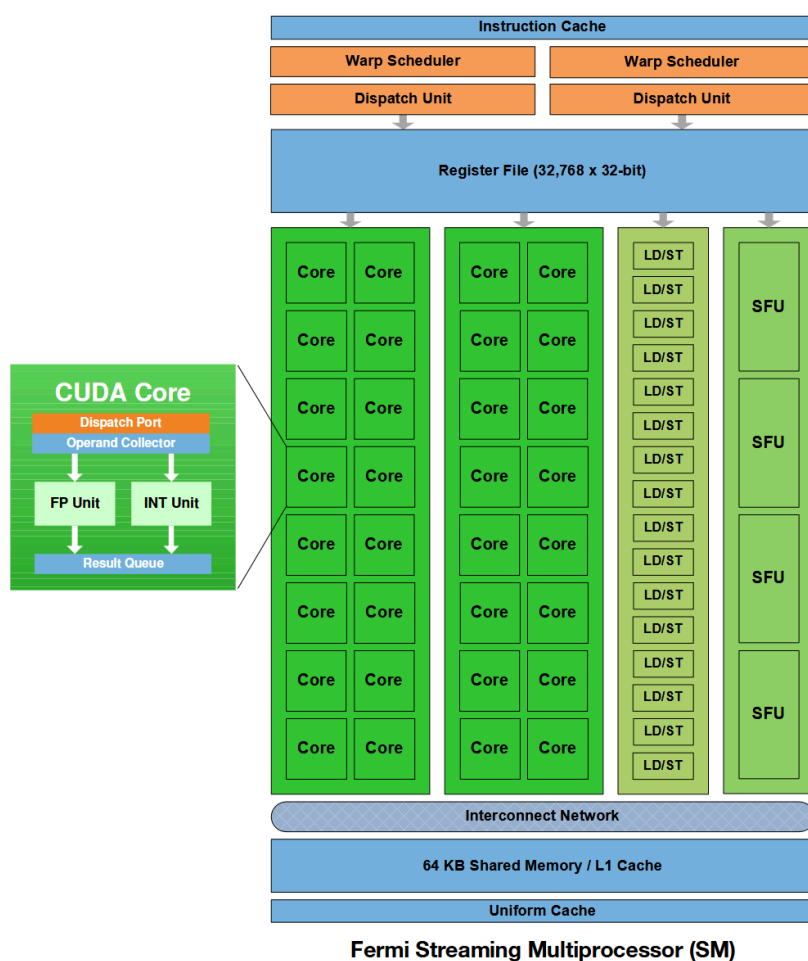


Рис. 4 — Поточковый мультипроцессор архитектуры Fermi (SM Fermi)

В потоковом мультипроцессоре на место SP появились CUDA-ядра. CUDA-ядро состоит из порта для блока распределения и выдачи задач, операнда-коллектора, вычислительных элементов и очереди для результатов выполнения поставленных задач ядру. Вычислительные элементы являются FP-вычислительные блоки (FPU), для вычисления чисел с плавающей точкой и INT-вычислительные блоки (ALU), для вычислений целочисленных чисел. Также в потоке мультипроцессора появился блок LD/ST (Load/Store), который является блоком загрузки и сохранения. Также в SM появился реги-

стовый файл (Register File), для сохранения промежуточных значений, планировщик (Warp Scheduler) и блок выдачи и распределения задач (Dispatch Unit) [8].

Теперь рассмотрим более современную микроархитектуру - Turing, разработанную Nvidia для видеокарт, начиная с 2018 года. Данная архитектура представляла собой первую микроархитектуру, которая поддерживала трассировку лучей в реальном времени и применялась в потребительских продуктах [9]. Одним из графических процессоров, разработанных на микроархитектуре Turing являлся графический процессор NVIDIA TU102. С его архитектурой можно ознакомиться на рис. 5:

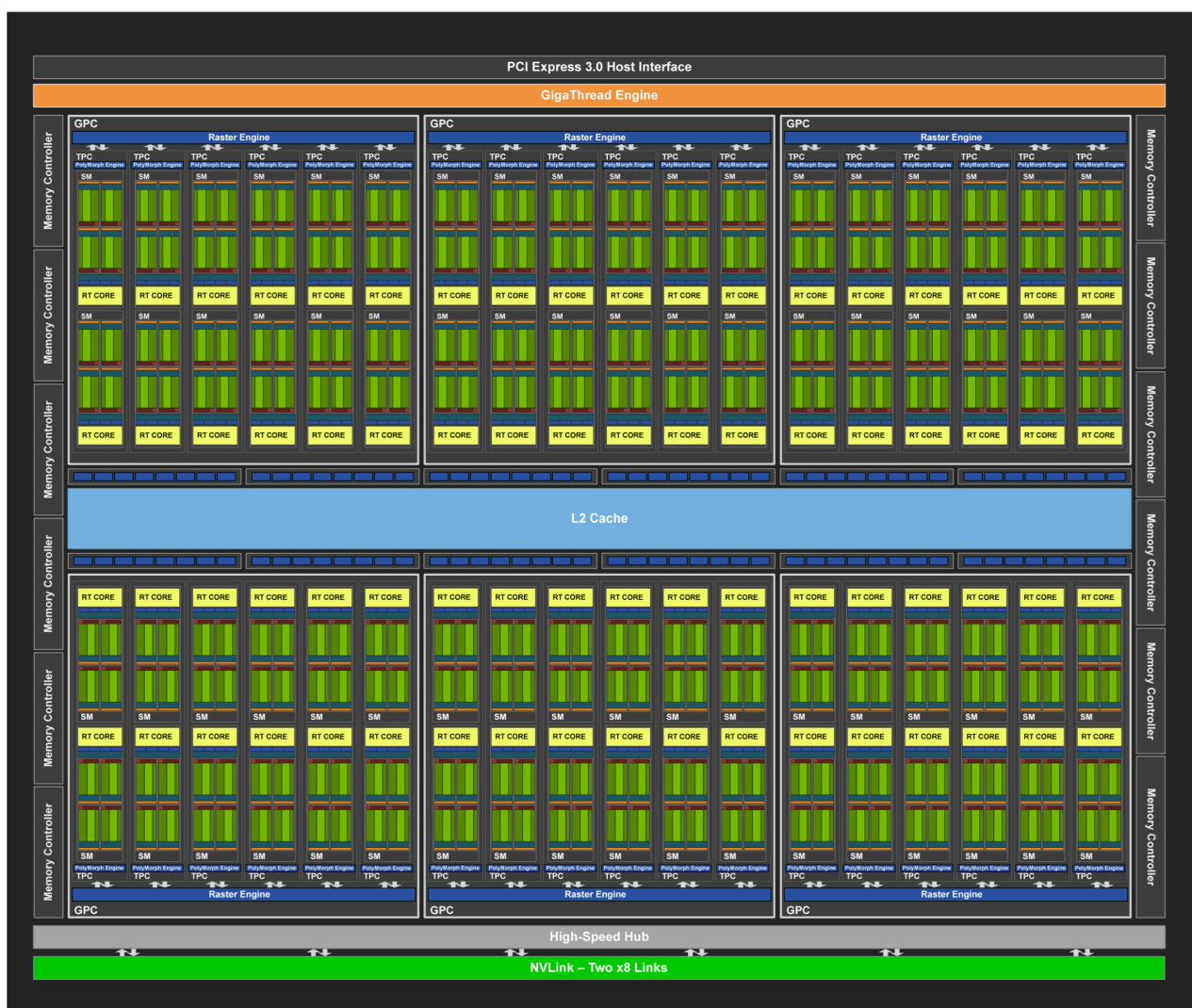


Рис. 5 — Архитектура графического процессора NVIDIA TU102

Такая архитектура является уже сильно улучшенной по сравнению с предыдущими представленными. За 8 лет, количество графических ядер у графических процессоров заметно увеличилось. На архитектуре графического процессора NVIDIA TU102 можно увидеть около 4608 универсальных ядер (CUDA-ядер), 72 ядер для выполнения рей-трейсинга и 576 тензорных ядер. Один графический процессор может иметь 6

вычислительных графических кластеров (GPC), в каждом из которых по 12 потоковых мультипроцессоров (SM). В каждом мультипроцессоре есть 1 ядро для выполнения рей-трейсинга и 4 вычислительных единицы, в каждом из которых есть по 16 вычислительных CUDA-ядер и 2 тензорных ядра. В новых потоковых мультипроцессорах помимо CUDA-ядер появились ядра для рей-трейсинга (RT Core) и тензорные ядра (Tensor Core) для обучения нейросетей [10]. Структуру потокового мультипроцессора (SM) Turing NVIDIA TU102 можно увидеть на рис. 6:

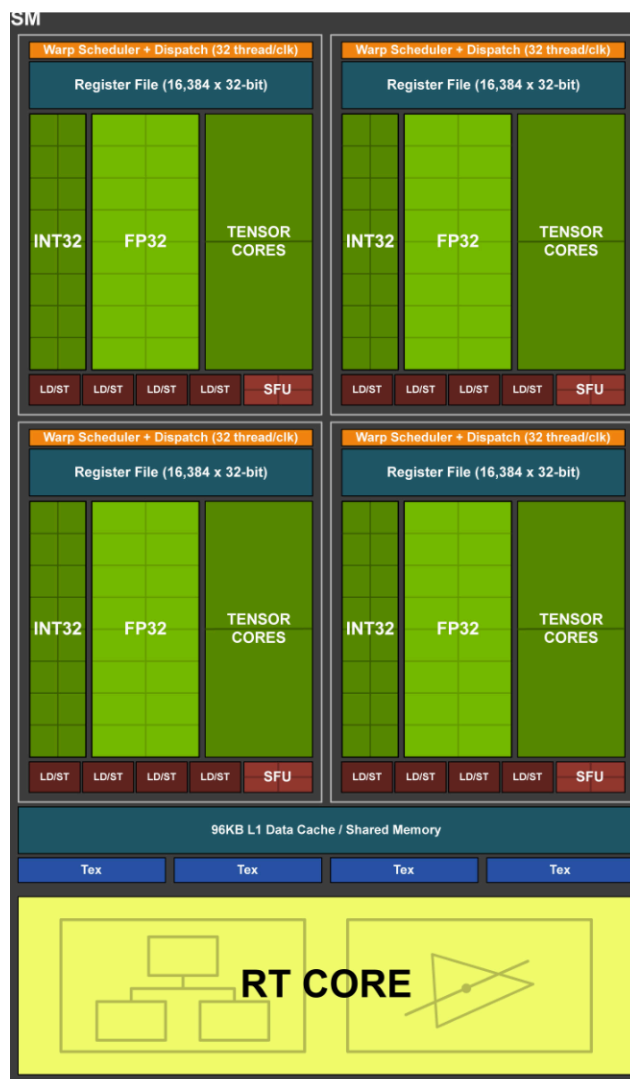


Рис. 6 — Архитектура SM NVIDIA TU102

Количество LD/ST-блоков знатно меньше, чем вычислительных элементов у потокового мультипроцессора. Если в архитектуре Fermi использовалось на 2 ядра 1 LD/ST блок, то количество ядер на 1 блок загрузки и сохранения увеличилось вдвое. В будущих архитектурах, структура потокового мультипроцессора в общем останется неизменной. В архитектуре Blackwell заметно общее увеличение SM-блоков и улучшение тензорных и рей-трейсинговых ядер.

2. Классификация параллельных вычислительных моделей (SIMT vs SIMD, MIMD).

Современные графические ускорители выполняют большое количество мелких задач одновременно. В отличие от центрального процессора (CPU), который предназначен для последовательного выполнения комплексных задач, графические ускорители спроектированы так, чтобы те выполняли множество небольших и независимых вычислений, таких как, к примеру отрисовки пикселей или обучение нейросетей. Одну такую мелкую задачу может выполнять специализированный блок, который называют ядром графического ускорителя. В современных видеокартах (GPU), количество ядер может скалироваться от нескольких тысяч до нескольких десятков тысяч.

Различие количества ядер у CPU и GPU является в том, что центральный процессор выполняет большой спектр различных, сложных и разнообразных задач, последовательно, в то время, как ядро GPU является узкоспециализированным ядром, который может выполнять лишь определенный список задач, которые приходят на обработку. Объем задач у CPU и GPU сильно отличается, ядро CPU обладает большим размером кеша, чем ядро GPU. Чем меньше логики у аппаратного блока и чем меньше для него создано памяти для хранения временных и локальных данных и инструкций, тем меньше размер аппаратного блока. Следовательно размер одного ядра CPU намного больше ядра GPU.

В отличие от ядер CPU, ядра GPU выполняют параллельно похожие операции над данными. В компьютерной графике, мультимедиа, криптографии и машинном обучении часто происходит ситуация, когда нужно работать с большим объемом данных, которым требуется одна и та же обработка. Взять в пример вычисление какой-либо картинки на экране монитора - на сегодняшний день, разрешения экранов достигают более 2 миллионов пикселей, что представляет собой уже огромный пласт работы, который необходимо обрабатывать чуть ли не за десятки миллисекунд. Поэтому, графические ускорители обладают маленькими, но многочисленными ядрами, тем самым позволяя работать с кучей данных, требующих одних и тех же инструкций за очень короткое время.

В разработке графических ускорителей принимают архитектурную классификацию параллелизма в потоках команд и данных SIMD, которая является одной из

основных возможных архитектур параллелизма, основанных на таксономии Флинна. Среди основных классов наблюдаются следующие виды:

- ОКОД (SISD - single instruction, single data) - одиночный поток команд, одиночный поток данных;
- ОКМД (SIMD - single instruction, multiply data) - одиночный поток команд, множественный поток данных;
- МКОД (MISD - multiply instruction, single data) - множественный поток команд, одиночный поток данных;
- МКМД (MIMD - multiply instruction, multiply data) - множественный поток команд, множественный поток данных [11].

Все классы архитектур по Флинну представлены на рис. 7:

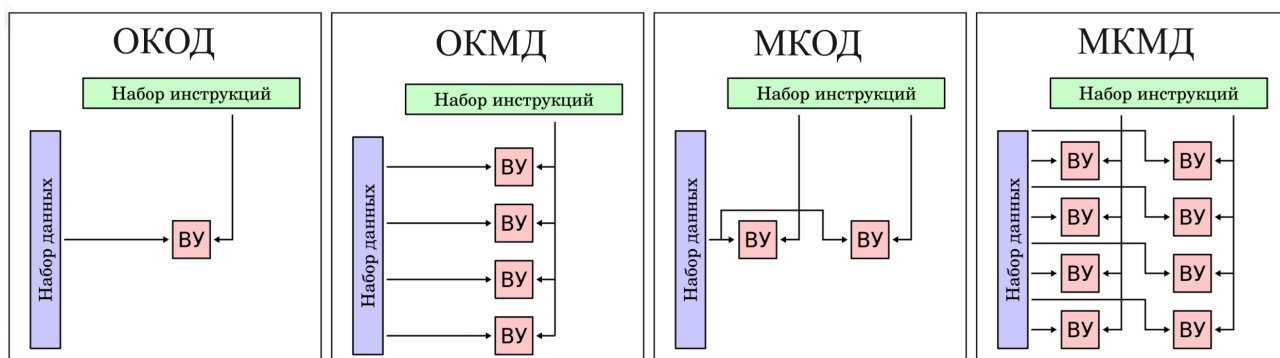


Рис. 7 — Все виды архитектур по таксономии Флинна

Примером архитектуры SISD является традиционный компьютер фон-Неймановской архитектуры с одним процессором, который выполняет последовательно одну инструкцию за другой, работая с одним потоком данных. SISD не является параллельной [12]. Основывать графический процессор на архитектуре SISD приведёт к слабой производительности, так как архитектура SISD вызывает ограничение скорости из-за своей природы. Первые процессоры основывались на этой архитектуре.

Архитектура SIMD позволяет обеспечить параллелизм на уровне данных, позволяя работать с большим количеством информации, над которой требуется работа лишь одной конкретной инструкции. [13] Технология SIMD не только улучшает производительность вычислительных систем, но и уменьшает площадь, ибо количество необходимых шин, для того чтобы связать аппаратные блоки существенно уменьшается. На основе этой архитектуры была разработана архитектура SIMT (Single Instruction Multiple Threads) - одна инструкция, множество потоков.

Архитектура MISD основана на том, что множество инструкций будет работать с одиночным потоком данных. Это может быть полезно в различных системах с отказоустойчивостью: в космонавтике или авионике. Множество процессоров работают над одним и тем же потоком данных. [14] В нейросетях и графике нет необходимости различными инструкциями выстраиваться в очередь для обработки одного потока данных. При такой архитектуре графическое ядро будет обладать большими задержками, что противоречит цели в создании производительной архитектуры графического ускорителя.

MIMD архитектура является распространенной архитектурой в компьютерах с несколькими процессорами. Благодаря такой архитектуре, можно обеспечить эффективное использование аппаратных ресурсов. При этом, ядра, которые будут использовать архитектуру MIMD будут сложными, т.к. на каждый поток команд и инструкций необходимо разработать блок выборки и декодирования команд, что увеличит место на кристалле, а также увеличит объем потребляемой энергии. Из-за такой архитектуры также необходимо разработать сложную систему планирования, потому что в MIMD могут возникнуть проблемы с взаимной блокировкой и состязанием за обладанием ресурсами. Это связано с тем, что потоки, пытаясь получить доступ к ресурсам, могут встретиться непредсказуемым способом на одном и том же ресурсе и такую проблему необходимо будет решать отдельными компонентами в архитектуре, которые также будут занимать место и энергию на кристалле [15].

Среди всех архитектур для разработки графического ядра явно подходит архитектура SIMD. На её основе будет разрабатываться архитектура графического ядра.

3. Архитектура ядра

Заключение

Литература

- [1] И.А. Дудкин, Д.М. Старухин, и В.В. Черкасов, «Обзор и анализ 2D графических ускорителей».
- [2] И.А. Дудкин, Д.М. Старухин, и В.В. Черкасов, «Разработка архитектуры аппаратной реализации 2D графического ускорителя».
- [3] «Видеокарта». [Онлайн]. Доступно на: <https://ru.wikipedia.org/wiki/%D0%92%D0%B8%D0%B4%D0%B5%D0%BE%D0%BA%D0%B0%D1%80%D1%82%D0%B0>
- [4] «Nvidia». [Онлайн]. Доступно на: <https://ru.wikipedia.org/wiki/Nvidia>
- [5] «NVIDIA NV30». [Онлайн]. Доступно на: <https://www.techpowerup.com/gpu-specs/nvidia-nv30.g20>
- [6] Э. С. Ершов и Е. А. Сухотерин, «Архитектура графических процессоров Nvidia восьмого поколения с точки зрения вычислений общего назначения».
- [7] «NVIDIA Tesla A Unified Graphics and Computing Architecture». [Онлайн]. Доступно на: <https://www.techpowerup.com/gpu-specs/docs/nvidia-tesla-architecture.pdf>
- [8] «NVIDIA's Next Generation CUDA Compute Architecture». [Онлайн]. Доступно на: <https://www.techpowerup.com/gpu-specs/docs/nvidia-fermi-compute-architecture.pdf>
- [9] «Turing (микроархитектура)». [Онлайн]. Доступно на: [https://en.wikipedia.org/wiki/Turing_\(microarchitecture\)](https://en.wikipedia.org/wiki/Turing_(microarchitecture))
- [10] «NVIDIA TURING GPU ARCHITECTURE». [Онлайн]. Доступно на: <https://www.techpowerup.com/gpu-specs/docs/nvidia-turing-architecture.pdf>
- [11] «Таксономия Флинна». [Онлайн]. Доступно на: https://en.wikipedia.org/wiki/Flynn%27s_taxonomy
- [12] «SISD». [Онлайн]. Доступно на: <https://ru.wikipedia.org/wiki/SISD>
- [13] «SIMD». [Онлайн]. Доступно на: <https://ru.wikipedia.org/wiki/SIMD>
- [14] «MISD». [Онлайн]. Доступно на: <https://ru.wikipedia.org/wiki/MISD>
- [15] «MIMD». [Онлайн]. Доступно на: <https://ru.wikipedia.org/wiki/MIMD>